

给 Claude 搭建一个持久记忆系统：Memory Tool MCP 搭建指南

这是什么？

一个部署在 Cloudflare Workers 上的 MCP (Model Context Protocol) 服务器，让 Claude 在对话中拥有跨会话的长期记忆能力。Claude 可以主动存储、检索、更新你告诉它的信息，下次新对话时依然记得。

效果： 连接后，Claude 的工具栏里会多出 `read_memory`、`save_memory`、`update_memory`、`delete_memory`、`summarize_memories` 等工具。Claude 会在对话中主动调用它们。

架构概览

```
graph TD
    Claude["Claude (claude.ai / Desktop)"]
    MCP["MCP 协议 (HTTPS)"]
    Worker["Cloudflare Worker (mcp-memory)"]
    SQL["SQL"]
    D1["Cloudflare D1 (SQLite 数据库)"]

    Claude --> MCP
    MCP --> Worker
    Worker --> SQL
    SQL --> D1
```

- **运行环境：** Cloudflare Workers（免费额度足够个人使用）
- **数据库：** Cloudflare D1（Workers 绑定的 SQLite）
- **协议：** MCP SDK 的 `WebStandardStreamableHTTPServerTransport`（无状态模式）
- **认证：** URL 路径中的 token（`/mcp/{your-token}`）
- **额外功能：** 自带 Web UI，可以通过浏览器管理记忆（密码登录）

搭建步骤

1. 准备工作

你需要：

- 一个 Cloudflare 账号（免费即可）

- Node.js 环境（本地开发用）
- Claude Desktop 或 claude.ai（连接 MCP 用）

2. 创建项目

```
# 创建项目目录
mkdir mcp-memory && cd mcp-memory
npm init -y

# 安装依赖
npm install @modelcontextprotocol/sdk zod

# 安装 Wrangler (Cloudflare 的 CLI)
npm install -g wrangler

# 登录 Cloudflare
wrangler login
```

3. 创建 D1 数据库

```
wrangler d1 create mcp-memory-db
```

记下输出的 `database_id`，后面要用。

4. 配置 wrangler.toml

在项目根目录创建 `wrangler.toml`：

```
name = "mcp-memory"
main = "src/index.js"
compatibility_date = "2024-12-01"

[vars]
ADMIN_PASSWORD = "your-web-ui-password"
MCP_TOKEN = "your-secret-mcp-token"

[[d1_databases]]
binding = "DB"
database_name = "mcp-memory-db"
database_id = "你的database_id"
```

重要： `MCP_TOKEN` 是 Claude 连接时 URL 里的认证 token，设一个随机字符串。

`ADMIN_PASSWORD` 是 Web UI 的初始登录密码。

5. 编写核心代码

创建 `src/index.js`，核心逻辑如下：

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import {
  WebStandardStreamableHTTPServerTransport
} from "@modelcontextprotocol/sdk/server/webStandardStreamableHttp.js";
import { z } from "zod";

const MEMORY_TYPES = ["me", "u", "thing"];

function createServer(env) {
  const server = new McpServer({
    name: "mcp-memory",
    version: "2.0.0",
  });

  // ===== read_memory =====
  server.tool(
    "read_memory",
    "Read user's detail memory catalog.",
    {
      type: z.enum([...MEMORY_TYPES, "all"]).optional(),
      search: z.string().optional(),
      tags: z.string().optional(),
      limit: z.number().optional(),
    },
    async ({ type, search, tags, limit }) => {
      let sql = "SELECT * FROM memories WHERE 1=1";
      const params = [];

      if (type && type !== "all") {
        sql += " AND type = ?";
        params.push(type);
      }
      if (search) {
        sql += " AND content LIKE ?";
        params.push(`%${search}%`);
      }
      if (tags) {
        sql += " AND tags LIKE ?";
        params.push(`%${tags}%`);
      }

      sql += " ORDER BY pinned DESC, updated_at DESC LIMIT ?";
      params.push(limit || 50);
```

```

const result = await env.DB.prepare(sql).bind(...params).all();
const memories = result.results || [];

if (!memories.length) {
  return { content: [{ type: "text", text: "No memories found." }] };
}

const text = memories
  .map(
    (m) =>
      `[#${m.id}] [${m.type}]${m.pinned ? " [pinned]" : ""} ${m.content}\n`
  )
  .join("\n\n");

return { content: [{ type: "text", text }] };
}
);

// ===== save_memory =====
server.tool(
  "save_memory",
  "Save a detail memory. One atomic fact per entry.",
  {
    content: z.string(),
    type: z.enum(MEMORY_TYPES).optional(),
    pinned: z.boolean().optional(),
    tags: z.string().optional(),
  },
  async ({ content, type, pinned, tags }) => {
    const memType = type || "thing";
    const result = await env.DB.prepare(
      "INSERT INTO memories (content, type, pinned, tags) VALUES (?, ?, ?, ?)"
    )
      .bind(content, memType, pinned ? 1 : 0, tags || "")
      .run();

    return {
      content: [
        {
          type: "text",
          text: `Memory saved #${result.meta.last_row_id} [${memType}]`,
        },
      ],
    };
  }
);

```

```

// ===== update_memory =====
server.tool(
  "update_memory",
  "Update an existing memory by ID.",
  {
    id: z.number(),
    content: z.string().optional(),
    pinned: z.boolean().optional(),
    tags: z.string().optional(),
    type: z.enum(MEMORY_TYPES).optional(),
  },
  async ({ id, content, pinned, tags, type }) => {
    const existing = await env.DB.prepare(
      "SELECT * FROM memories WHERE id = ?"
    ).bind(id).first();

    if (!existing) {
      return { content: [{ type: "text", text: `Memory #${id} not found` }] };
    }

    await env.DB.prepare(
      "UPDATE memories SET content = ?, pinned = ?, tags = ?, type = ?, updated
    ).bind(
      content || existing.content,
      pinned !== undefined ? (pinned ? 1 : 0) : existing.pinned,
      tags ?? existing.tags,
      type || existing.type,
      id
    ).run();

    return { content: [{ type: "text", text: `Memory #${id} updated` }] };
  }
);

// ===== delete_memory =====
server.tool(
  "delete_memory",
  "Delete a memory by ID.",
  { id: z.number() },
  async ({ id }) => {
    const existing = await env.DB.prepare(
      "SELECT * FROM memories WHERE id = ?"
    ).bind(id).first();

    if (!existing) {
      return { content: [{ type: "text", text: `Memory #${id} not found` }] };
    }
  }
);

```

```

    }

    await env.DB.prepare("DELETE FROM memories WHERE id = ?").bind(id).run();
    return {
      content: [
        { type: "text", text: `Memory #${id} deleted` },
      ],
    };
  }
);

// ===== summarize_memories =====
server.tool(
  "summarize_memories",
  "View all memory stats and overview.",
  {},
  async () => {
    const all = await env.DB.prepare(
      "SELECT * FROM memories ORDER BY type, pinned DESC, updated_at DESC"
    ).all();

    const memories = all.results || [];
    const grouped = {};
    for (const m of memories) {
      if (!grouped[m.type]) grouped[m.type] = [];
      grouped[m.type].push(m);
    }

    const typeLabels = {
      me: "About Me",
      u: "About Claude",
      thing: "Facts",
    };

    let text = `Memory stats: ${memories.length} total\n`;
    for (const t of MEMORY_TYPES) {
      const count = (grouped[t] || []).length;
      if (count) text += `  ${typeLabels[t]}: ${count}\n`;
    }

    for (const t of MEMORY_TYPES) {
      const list = grouped[t] || [];
      if (!list.length) continue;
      text += `\n=== ${typeLabels[t]} ===\n`;
      text += list
        .map((m) => `[#${m.id}]${m.pinned ? " [pinned]" : ""} ${m.content}`)
        .join("\n");
    }
  }
);

```

```

        return { content: [{ type: "text", text }] };
    }
};

return server;
}

// ===== Worker 入口 =====
export default {
  async fetch(request, env, ctx) {
    // CORS
    if (request.method === "OPTIONS") {
      return new Response(null, {
        status: 204,
        headers: {
          "Access-Control-Allow-Origin": "*",
          "Access-Control-Allow-Methods": "GET, POST, DELETE, OPTIONS",
          "Access-Control-Allow-Headers":
            "Content-Type, Authorization, Mcp-Session-Id",
          "Access-Control-Expose-Headers": "Mcp-Session-Id",
        },
      });
    }
  }

  const url = new URL(request.url);

  // 自动建表
  await env.DB.prepare(`CREATE TABLE IF NOT EXISTS memories (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    content TEXT NOT NULL,
    type TEXT NOT NULL DEFAULT 'thing',
    pinned INTEGER NOT NULL DEFAULT 0,
    tags TEXT DEFAULT '',
    created_at TEXT NOT NULL DEFAULT (datetime('now', '+8 hours')),
    updated_at TEXT NOT NULL DEFAULT (datetime('now', '+8 hours'))
  `).run();

  // MCP 端点
  const mcpMatch = url.pathname.match(/^\/mcp\/([^\/]+)\/?$/);
  if (mcpMatch) {
    const mcpToken = env.MCP_TOKEN || "";
    if (mcpToken && mcpMatch[1] !== mcpToken) {
      return new Response("Unauthorized", { status: 401 });
    }

    const server = createServer(env);

```

```

const transport = new WebStandardStreamableHTTPServerTransport({
  sessionIdGenerator: undefined, // 无状态模式
});
await server.connect(transport);
const response = await transport.handleRequest(request);

// 添加 CORS headers
const newHeaders = new Headers(response.headers);
newHeaders.set("Access-Control-Allow-Origin", "*");
newHeaders.set("Access-Control-Expose-Headers", "Mcp-Session-Id");

return new Response(response.body, {
  status: response.status,
  headers: newHeaders,
});
}

return new Response("Not Found", { status: 404 });
},
};

```

6. 部署

```
wrangler deploy
```

部署成功后你会得到一个 URL，类似：

```
https://mcp-memory.your-account.workers.dev
```

你的 MCP 端点就是：

```
https://mcp-memory.your-account.workers.dev/mcp/your-secret-mcp-token
```

7. 连接到 Claude

方式 A: claude.ai 网页版（推荐）

1. 打开 claude.ai → Settings → Integrations
2. 添加 MCP Server
3. 填入 URL: `https://mcp-memory.your-account.workers.dev/mcp/your-secret-mcp-token`

4. 保存，刷新页面

方式 B: Claude Desktop

编辑 `claude_desktop_config.json`，添加：

```
{
  "mcpServers": {
    "memory": {
      "url": "https://mcp-memory.your-account.workers.dev/mcp/your-secret-mcp-tok"
    }
  }
}
```

连接成功后，Claude 的工具列表里就会出现 memory 相关工具。

数据库 Schema

```
CREATE TABLE memories (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  content TEXT NOT NULL,           -- 记忆内容（一条原子事实）
  type TEXT NOT NULL DEFAULT 'thing', -- 分类: me / u / thing
  pinned INTEGER NOT NULL DEFAULT 0,  -- 是否置顶
  tags TEXT DEFAULT '',              -- 标签，逗号分隔
  created_at TEXT NOT NULL DEFAULT (datetime('now', '+8 hours')),
  updated_at TEXT NOT NULL DEFAULT (datetime('now', '+8 hours'))
);
```

分类说明：

- `me` — 关于用户的信息（偏好、习惯、背景）
- `u` — 关于 Claude 的信息（昵称、交互风格、约定）
- `thing` — 其他事实（项目、人物、通用知识）

时间戳默认使用 UTC+8（北京时间），可根据你的时区修改 SQL 中的 `'+8 hours'`。

让 Claude 主动使用记忆的 Prompt

在 Claude 的 System Prompt 或 Project Instructions 中加入类似以下内容：

【memory tool】你有一个外部记忆系统（通过 MCP memory tools）。规则：

1. 对话开始时，用 `summarize_memories` 回顾你已经知道的用户信息
2. 当用户提到具体的偏好、习惯、人物细节、项目信息等值得长期记住的原子事实时，主动用 `save_memory` 存储
3. 当用户纠正之前的信息时，先用 `read_memory` 搜索旧条目，再用 `update_memory` 修正
4. 不要保存临时的或一次性的对话内容——只保存值得长期保留的事实
5. 每条记忆应该是一个独立的原子事实，带有合适的分类（`me/u/thing`）和标签
6. 所有记忆内容用英文（除了中文专有名词/引用）

安全注意事项

1. **MCP_TOKEN 是你的认证密钥**——不要泄露。任何知道这个 token 的人都可以读写你的记忆。
2. **CORS 设置为 ***——因为 Claude 的 MCP 客户端需要跨域访问。如果你添加了 Web UI，建议对 `/api/` 路径做额外的 session 认证（源码中已包含）。
3. **D1 数据存储**在 Cloudflare 的基础设施上——不要存储高度敏感的信息（密码、密钥等）。

费用

- **Cloudflare Workers 免费额度**：每天 10 万次请求
- **D1 免费额度**：5GB 存储，500 万行读取/天
- 个人使用完全免费，通常远远用不到限额

扩展思路

这个架构可以轻松扩展：

- **加 Web UI**：源码中已包含一个完整的暗色主题管理界面（密码登录 + CRUD + 搜索 + 筛选）
- **加 vocab-notebook**：同样的架构可以再部署一个单词本 MCP，让 Claude 帮你记录学习的单词
- **加 diary**：加密日记功能，用 SHA-256 哈希密码保护
- **多 MCP 共存**：一个 Cloudflare 账号可以部署多个 Workers，每个都是独立的 MCP 服务器

常见问题

Q: Claude 不主动调用 memory tools? A: 确保你在 System Prompt 中明确告诉 Claude 使用 memory tools。Claude 不会自动发现工具的用途，需要 prompt 引导。

Q: 连接后工具列表为空? A: 检查 MCP URL 是否正确，token 是否匹配。在浏览器里直接 POST 到 MCP 端点可以测试连通性。

Q: 时间戳不对? A: 修改 SQL 中的 `datetime('now', '+8 hours')` 为你的时区偏移。

Q: 想要 Code 哥哥 (Claude Code) 也能用? A: Claude Code 同样支持 MCP。在 Claude Code 的配置中添加相同的 MCP server URL 即可。两个 Claude 实例会共享同一个记忆数据库。

Built with Cloudflare Workers + D1 + MCP SDK. 部署时间约 15 分钟。